

CustomVRAD

Technical Report: Lightmap Baking Algorithms

Path-traced and radiosity pipelines for Source SDK 2013 / Garry's Mod

CustomVRAD contributors

August 6, 2026

Abstract

CustomVRAD is a 64-bit lightmap compiler derived from Valve VRAD (Source SDK 2013) for Garry's Mod. It retains the classical patch-radiosity pipeline and optionally replaces world-face (and static-prop) lighting with a DirectX Raytracing (DXR) path tracer. Extensions include hero-wavelength spectral transport, contact-hardening soft shadows, textured emissive and filter materials, spatial volume overrides, multi-scale ambient occlusion, IESNA photometric spot evaluation, directional projected-texture modulation on `light_spot`, and lightmap post-processes (denoising, seam stitching, edge pull). This document states the algorithms as implemented: estimators, coordinate conventions, parameter semantics, and the relationship between classic and path-traced bake modes.

Contents

1	System overview	3
1.1	Data flow	3
1.2	Notation	3
2	Path tracing (DXR)	3
2.1	PathLi estimator	4
2.2	Bounce semantics	4
2.3	Next-event estimation	4
2.4	Soft shadows and contact hardening	5
2.5	Firefly clamp	5
2.6	Sampling and luxel AA	5
2.7	Post-bake lightmap cleanup	6
3	Spectral transport	6
3.1	Hero wavelengths	6
3.2	RGB $\leftrightarrow$ spectrum	6
4	Classic radiosity	6
4.1	Patches and form factors	7
4.2	Gather iteration	7
4.3	Textured bounce (<code>-texbounce</code>)	7
4.4	Cavity-damped energy (<code>-energy</code> / <code>-cavity</code>)	7
4.5	Artistic bounce controls	7

4.6	Radial splat and bounce weld	8
4.7	OpenCL gather (-gpu)	8
5	Materials	8
5.1	Emissive surfaces (\$vrad_emit*)	8
5.2	Colored glass filters (\$vrad_filter*)	8
6	Oklab perceptual ops	8
7	Volume entities	9
7.1	light_env_vol	9
7.2	light_absorb	9
7.3	light_bounce_vol	9
7.4	light_volume	9
8	Ambient occlusion	9
9	Soft sun	10
10	Photometric spots and projected textures	10
10.1	IESNA photometry	10
10.2	Projected texture classification	10
10.3	Planar framing in the outer cone	11
10.4	Cubemap direction mapping	11
10.5	Composite intensity	11
11	Denoising	11
11.1	OIDN (default)	11
11.2	OptiX	12
11.3	Sakai (statistical)	12
12	Static prop lighting	12
12.1	Classic speedups	12
12.2	Pathtrace props	12
13	Lightmap sample quality	12
13.1	Edge pull	12
13.2	Seam stitching	12
14	Reference integrator (pseudocode)	13
15	Parameter cheat sheet	13
16	Implementation map	14

1 System overview

CustomVRAD produces HDR and LDR lightmaps compatible with the Source engine. World-face lighting uses exactly one of two pipelines:

1. **Classic radiosity** — stock VRAD BuildFacelights and iterative patch gather, optionally accelerated by OpenCL (`-gpu`).
2. **Path tracing** — D3D12 DXR baker (`-pathtrace` / `-dxr`). On success it replaces world-face direct lighting and radiosity bounce. Static props may share the integrator under `-StaticPropLighting`.

Leaf ambient cubes, detail props, and engine-exported worldlights remain on stock paths, with volume-aware CustomVRAD extensions where noted. If DXR initialisation or bake fails, the compiler falls back to classic radiosity for world faces (and CPU gather for props).

GPU backends

`-pt_gpu` selects the DXR RayQuery luxel baker. `-gpu` selects an independent OpenCL path for classic radiosity gather and batched AO occlusion. The two backends are not interchangeable.

1.1 Data flow

1. Load BSP, materials (VMT/VTF), entities, static props.
2. Build ray-tracing acceleration (CPU KD-tree and/or DXR TLAS/BLAS).
3. Direct lighting (classic) *or* PathLi bake (pathtrace).
4. Radiosity bounce (classic only, if `-bounce N > 0`).
5. Optional AO, denoise, seam stitch, edge-pull-aware sampling.
6. Write lightmaps / prop vertex colours into the BSP.

1.2 Notation

Symbol	Meaning
$\mathbf{x}, \mathbf{n}$	Sample position and shading normal
ρ	Diffuse albedo (linear RGB or spectral reflectance)
β	Path throughput
L_e	Emitted radiance
V	Binary (or soft) visibility
N	<code>-pt_bounces</code> — number of <i>indirect</i> hops
M	Path depth = $N + 1$ (primary vertex + N bounces)

2 Path tracing (DXR)

The path tracer is a unidirectional Lambertian integrator with next-event estimation (NEE) at every path vertex. It is implemented twice:

- **GPU** (`-pt_gpu`, default): compute shader using DXR 1.1 RayQuery, TLAS over world + optional prop BLASes (`pathtrace_dxr_gpu_bake.inl`).

- **CPU** (`-pt_cpu`): threaded SSE reference (`PtPathLi` in `pathtrace_dxr.cpp`). RGB-only; no spectral.

2.1 PathLi estimator

At luxel (or prop sample) $(\mathbf{x}_0, \mathbf{n}_0)$ the estimator is

$$\hat{L} = \sum_{k=0}^m \beta_k L_{\text{NEE}}(\mathbf{x}_k, \mathbf{n}_k) + \beta_{m'} L_{\text{sky}}(\mathbf{x}_{m'}), \quad (1)$$

where m is the last surface vertex before termination, and the sky term is added when a BSDF sample escapes to the sky (ambient sky is *not* NEE-sampled).

BSDF. Surfaces are Lambertian,

$$f_r(\omega_i, \omega_o) = \frac{\rho}{\pi}, \quad p_{\text{bsdf}}(\omega) = \frac{\cos \theta}{\pi}. \quad (2)$$

After a non-sky hit the throughput update simplifies to

$$\beta_{k+1} = \beta_k \rho_k \quad (\text{albedo clamped channelwise to } [0, 1]). \quad (3)$$

Russian roulette. Starting at bounce index $k \geq 1$,

$$q = \min(0.95, \max(\beta_r, \beta_g, \beta_b)) \quad (\text{spectral: mean of four } \beta_\lambda). \quad (4)$$

Terminate if $u > q$; otherwise $\beta \leftarrow \beta/q$.

2.2 Bounce semantics

$\text{pathDepth} = N + 1, \quad N = \text{-pt_bounces}$

N counts **indirect hops after the luxel**. Vertex $k = 0$ is always the receiving sample (primary NEE = direct lighting). Therefore:

N	Path depth	Effect
0	1	Direct lights + sky escape; no GI
1	2	One bounce of reflected light
6	7	Typical indoor quality (config sheet)

Defaults: $N = 3$ when unset; $N = 1$ under **-fast**; clamp $[0, 16]$. Unset is distinguished from explicit zero by an internal sentinel (-1). **-pt_prop_bounces** mirrors this for vertex prop quality (auto = world N).

2.3 Next-event estimation

At every vertex the integrator evaluates (or samples) lights:

- **Sun / sky disc** — always evaluated; Veach power-heuristic MIS vs. cosine BSDF for the angular sun.

- **Point / spot** — all lights when `-pt_lights 0`; otherwise power-proportional CDF sample of K locals with weight $1/(p_i K)$. Bounce 0 always evaluates all locals on GPU.
- **\$vrad_emit_area_triangles** — power-weighted triangle selection + uniform area sample (PBRT-style).
- **Legacy emit_surface** — NEE-only proxies (no BSDF hit contribution; avoids double-counting).

Sun MIS. For sun solid angle $\Omega = 2\pi(1 - \cos \theta_{\max})$,

$$p_{\text{light}} = \frac{1}{\Omega}, \quad w_{\text{light}} = \frac{p_{\text{light}}^2}{p_{\text{light}}^2 + p_{\text{bsdf}}^2}. \quad (5)$$

Point / spot attenuation.

$$A(d) = \frac{1}{c + \ell d + q d^2}, \quad (6)$$

with optional hard distance cap and quintic smootherstep fade $s(t) = t^3(6t^2 - 15t + 10)$.

Area emission contribution (triangle sample with density p_A):

$$L_{\text{area}} = L_e \frac{(\mathbf{n}_r \cdot \boldsymbol{\omega})(-\mathbf{n}_e \cdot \boldsymbol{\omega}) V}{d^2 p_A}. \quad (7)$$

2.4 Soft shadows and contact hardening

Local lights may use a soft disk of radius r (`-pt_lightradius`, or entity `light_volume Radius`). The sun uses an angular cap (`-softsun` / entity `SunSpreadAngle`).

Adaptive sample count (CHSS-style) from a probe blocker distance t_{hit} and light distance d :

$$w_{\text{pen}} = \frac{t_{\text{hit}} r s_p}{\max(d - t_{\text{hit}}, 10^{-3})}, \quad n = \text{clamp}(1 + \lfloor 0.35 w_{\text{pen}} + 0.5 \rfloor, 1, n_{\max}), \quad (8)$$

where $s_p = \text{-pt_lightpenumbra}$ (default 1) and $n_{\max} = \text{-pt_softsamples}$ (default 16, clamp 1–64). Unoccluded lights use a cheap angular estimate, capped at four rays.

`-pt_softmode direct` (default) enables soft sampling only at the luxel; `all` enables it at every path vertex.

2.5 Firefly clamp

Hue-preserving max-channel clamp with cap $C_{\max} = 2500$:

$$C' = \begin{cases} C \cdot C_{\max} / \max(C) & \text{if } \max(C) > C_{\max}, \\ C & \text{otherwise.} \end{cases} \quad (9)$$

2.6 Sampling and luxel AA

- **-pt_samples S** : spp per luxel (quality-derived if unset; clamp 1–4096). GPU uses fixed S ; CPU may adaptively raise spp (up to $4\times$, cap 64) when coefficient of variation is high.
- **-pt_aa A** : deterministic $A \times A$ footprint grid over the luxel ($A \in [1, 5]$, default 3). Spatial AA, not a multiplicative spp loop.

2.7 Post-bake lightmap cleanup

After PathLi accumulation the baker:

1. Fills chart holes / invalid samples.
2. Dilates suspicious dark edge outliers (sample-in-solid).
3. Optionally denoises (section 11).
4. Always applies two FXAA-like 3×3 high-contrast evening passes (blend starts near 25% relative luminance jump, full near 55%, neighbour blend capped at 65%).

3 Spectral transport

Default on GPU: `-pt_spectral`. Disable with `-pt_nospectral`. CPU PathLi remains linear RGB.

3.1 Hero wavelengths

Four stratified wavelengths per path over [380, 780] nm:

$$\lambda_i = 380 + (i + \xi_i) \frac{400}{4}, \quad i = 0, 1, 2, 3. \quad (10)$$

3.2 RGB $\leftrightarrow$ spectrum

- **Upsampling:** Smits (1999) bases as tabulated in PBRT-v3 (32 samples over the visible range) — separate bases for reflectance and illuminants.
- **Transport:** illumination adds and reflectance multiplies in λ -space.
- **Projection:** Wyman analytic CIE 1931 2° CMFs $\rightarrow$ XYZ, then IEC 61966-2-1 / Rec. 709 matrix to linear sRGB.

Monte Carlo reconstruction weight uses the CIE Y integral $Y_{\text{int}} = 106.856895$:

$$(X, Y, Z) += \frac{L(\lambda)}{p_\lambda Y_{\text{int}}} (\bar{x}(\lambda), \bar{y}(\lambda), \bar{z}(\lambda)). \quad (11)$$

Spectral mode is implemented in the GPU HLSL baker only. `-pt_cpu` ignores `-pt_spectral` and transports in linear RGB.

4 Classic radiosity

Used when pathtrace is off or fails. `-bounce 0` skips the entire bounce stage (direct only) — the classic analogue of `-pt_bounces 0`.

4.1 Patches and form factors

Faces are subdivided into patches (unless bounce count is zero). Visibility is PVS-pruned and ray-tested. Far-field differential form factor:

$$F_{1\leftrightarrow 2} = \frac{(-\hat{d} \cdot \mathbf{n}_1)(\hat{d} \cdot \mathbf{n}_2)}{\|\mathbf{x}_1 - \mathbf{x}_2\|^2}. \quad (12)$$

Polygon-to-differential form factors are used when patch size invalidates the far-field approximation. Raw transfer = source area $\times$ form factor. Receiver lists are normalised:

$$s = \begin{cases} 1/T & T > \pi, \\ 1/\pi & T \leq \pi, \end{cases} \quad T = \sum_j t_j. \quad (13)$$

4.2 Gather iteration

Emitted light E_j and patch reflectance ρ_j gather as

$$A_i = \sum_j E_j \odot \rho_j t_{ij}. \quad (14)$$

Parent patches are area-weighted averages of children. Iteration stops after **-bounce** rounds (default 100) or when added energy falls below $\max(1, 0.0015 E_{\text{bounce } 1})$.

4.3 Textured bounce (**-texbounce**)

Samples **\$basetexture** at each leaf-patch origin for colour bleed. Reflectance is clamped to $[0, 0.99]$. Supports VTF 7.5; falls back to flat texdata average.

4.4 Cavity-damped energy (**-energy** / **-cavity**)

Opt-in. Enclosure estimate $e = \min(1, T/\pi)$ yields

$$d_E = 1 - e^2(1 - C), \quad d = 1 + (d_E - 1)\alpha, \quad (15)$$

where $C \in [0.25, 1]$ is the fully enclosed scale (default 0.70) and α is the global or volume **EnergyMode** blend. Open areas stay near stock; closed rooms recirculate less.

4.5 Artistic bounce controls

- **-bounce_boost** B : scales *integrated bounce only* after all iterations ($B \in [0, 16]$; direct unchanged).
- **-bounce_chroma** C : Oklab saturation of gathered bounce, strongest on early bounces:

$$(a, b) \leftarrow (a, b) \left(1 + \frac{C}{1+k}\right), \quad k = 0\text{-based bounce index}, \quad (16)$$

keeping perceptual lightness L . Requires **-texbounce**.

Pathtrace ignores **-bounce_chroma** (non-physical). Bounce volumes may still apply boost to pathtrace albedo/throughput.

4.6 Radial splat and bounce weld

Bounce is written to luxels with radial kernel radius scaled by `-bounce_soft` (default 1, range 0.5–4).

`-bounce_weld` (off by default) restricts coplanar neighbour-face GI to a shared-edge band ($\text{normal} \cdot \geq 0.985$; falloff beyond ~ 24 patch radii). Reduces rectangular GI smear across coplanar V BSP splits; can blotch ceilings.

4.7 OpenCL gather (`-gpu`)

Median-split BVH (≤ 8 tris/leaf). Kernels batch visibility and non-bump bounce gather over a CSR-like transfer graph (same $E_j \rho_j t_{ij}$ math). Bump gathers stay on CPU. Default batch 32768 rays; errors fall back to CPU.

5 Materials

5.1 Emissive surfaces (`$vrad_emit*`)

Enabled by `$vrad_emit 1` or `$vrad_emitstrength > 0` (default strength 200 when flag-only). Colour from `$vrad_emitmap/$vrad_emissivemap`, else `$basetexture`; optional grayscale `$vrad_emitmask`.

Texture RGB is linearised approximately as $x^{2.2}$, then scaled by strength and `lightscale`. Pathtrace builds a fan-triangulated mesh and uses area NEE; `-pt_emit_samples 0` evaluates all triangles (default 64 power samples). Classic mode aggregates to a softened area proxy plus a later self-illumination pass.

5.2 Colored glass filters (`$vrad_filter*`)

Pathtrace-only thin-sheet transmission. Ordinary `$translucent` glass does *not* tint light unless `$vrad_filter 1`.

Transmittance construction.

1. Sample filter map (or `basetexture`); square RGB for linearisation.
2. Thickness: $T \leftarrow T^{\text{thickness}}$ (Beer-like artistic).
3. Strength: Oklab lerp white $\rightarrow$ filter.
4. Opacity: multiply by $(1 - \text{opacity})$.

Rays may traverse up to 64 filter hits, deduplicating brush-sheet entrance/exit pairs within 12 world units. Default two-sided; `$vrad_filter_onesided 1` = front only.

Stacked panes (Oklab). Energy transport stays linear; stacking is perceptual:

$$L' = L_0 L_1, \quad a' = a_0 + a_1, \quad b' = b_0 + b_1. \quad (17)$$

6 Oklab perceptual ops

Björn Ottosson’s Oklab is used wherever artistic colour mixing must stay perceptually coherent. Linear sRGB $\leftrightarrow$ Oklab follows the standard LMS cube-root pipeline (coefficients in `oklab.h`).

Use site	Operation
Filter strength	Oklab lerp(white, T , strength)
Stacked filters	L multiply, a, b add
-bounce_chroma	Scale a, b ; keep L
Env-vol inbound tint	Keep light L ; take tint a, b

NEE intensities and $\rho \times L$ products remain in linear RGB (or λ).

7 Volume entities

All brush volumes share soft AABB membership: soft-min signed distance, quintic smootherstep blend shell (**BlendDistance** / **BlendMode**), and priority resolution (higher wins; ties $\rightarrow$ smaller AABB). Neighbour volumes use a soft Voronoi split so outside halos do not tint each other.

7.1 light_env_vol

Local sky / sun / ambient override. Volume lights are bake-only (not exported as engine worldlights). Leaf ambient cubes accumulate volume-blended sun when samples see the sun through sky. Shadow filters use the hard AABB via **OutsideCastShadowIn** / **InsideCastShadowOut**.

Optional **BounceVolColor** / **BounceVolBright** recolour bounced light inside the core to the volume's **_light** hue.

7.2 light_absorb

Direct multiplier $1 - w$ when **AbsorbDirect**; bounce multiplier $1 - \max(w_e, w_r)$ when **AbsorbBounce** (default on; strength default 0.5).

7.3 light_bounce_vol

Local overrides for **BounceBoost**, **BounceChroma**, and **EnergyMode**, soft-blended with CLI globals.

7.4 light_volume

Soft sphere point light: low-discrepancy samples inside radius R (default 16). Sample count scales with R (~ 16 at default, cap 40; reduced with **-fast**). Early-out when fully lit/shadowed. Exported as a hard point light at the centre (softness is bake-only).

8 Ambient occlusion

Multi-scale hemisphere AO in **FinalLightFace**, enabled by **-ao** / **-ao_*** or entities **light_ao** / **light_ao_vol**.

$$AO(\mathbf{x}) = \frac{\sum_i w_i V_i}{\sum_i w_i}, \quad w_i = \max(\mathbf{n} \cdot \omega_i, 0.08). \quad (18)$$

Directions with $\mathbf{n} \cdot \omega < -0.25$ are rejected (small wrapped hemisphere for contact darkening). Ray origin:

$$\mathbf{x}_0 = \mathbf{x} + \mathbf{n} b + \omega \max(0.05, 0.5b). \quad (19)$$

Final multiplier $M = \max(0, 1 - S + S \cdot AO)$ with strength $S \in [0, 8]$.

Defaults: samples 16, distance 48, strength 1, bias 0.25. Optional bilateral luxel denoise:

$$w = \exp\left[-\frac{\|\Delta x\|^2}{2\sigma_s^2} - \frac{(AO_j - AO_i)^2}{2(0.12)^2}\right]. \quad (20)$$

With `-gpu`, AO can use batched OpenCL occlusion.

9 Soft sun

Replaces stock's fixed 30 random cone rays:

- Low-discrepancy golden-angle spiral in the sun cone.
- Adaptive count ~ 9 at 2° up to ~ 40 at 50° .
- Early-out after a short probe when fully lit or fully shadowed.

0° = hard sun. Typical soft look: 0.5° – 3° .

10 Photometric spots and projected textures

Bake-only extensions on stock `light_spot`. Engine-exported worldlights retain cone keys for dynamic lighting; lightmap evaluation may substitute an IESNA intensity table and/or a projected Valve Texture Format (VTF) multiplier.

10.1 IESNA photometry

When the entity key `IES` names an LM-63 file under `garrysmo/IES/` (or a per-map subdirectory `IES/maps/mapname/`), the angular candela table replaces `_inner_cone`, `_cone`, and `_exponent` for the bake. Only `TILT=NONE` profiles are supported. Distance attenuation (`_constant_attn`, ...) is unchanged.

Let $\mathbf{b}$ be the beam axis and $\mathbf{r}$ a right vector from entity angles. For a unit direction ω from light to sample, vertical and horizontal angles (θ, ϕ) index a peak-normalized table $I(\theta, \phi) \in [0, 1]$. The photometric weight is $w_{\text{IES}} = s I(\theta, \phi)$ with scale $s = \text{IESScale}$ (default 1). Optional `IESBrightness` ≥ 0 overrides stock `_light` brightness for the bake only; `IESMaxIntensity` caps per-sample RGB after attenuation.

10.2 Projected texture classification

The key `ProjectedTexture` names a material under `materials/` (optional `materials/` prefix and `.vtf` suffix are stripped). The loader classifies the VTF:

Kind	Detection	Domain
Planar (2D)	No cubemap / no usable six-face layout	Spot cone
Cubemap	Six faces (<code>IsCubeMap</code> / <code>ENVMAP</code> + face count)	Full sphere
Spherical 2D	<code>ENVMAP</code> without six faces	Full sphere

Spherical 2D maps use equirectangular sampling when width $\geq \frac{3}{2}$ height; otherwise an OpenGL-style spheremap / matcap projection. Cubemap VTFs (including VTF 7.5 with CRC resources) are decoded to RGBA face arrays; GPU upload stores six consecutive `Texture2DArray` layers per cubemap in Source `CUBEMAP_FACE_*` order ($+X, -X, +Y, -Y, +Z, -Z$).

10.3 Planar framing in the outer cone

For planar projections, let $\alpha = \arccos(\text{_cone})$ (outer half-angle) and ω the light-to-sample direction. With $\cos \theta = \omega \cdot \mathbf{b}$:

$$u' = \frac{\omega \cdot \mathbf{r}}{\cos \theta \tan \alpha}, \quad v' = \frac{\omega \cdot \mathbf{u}}{\cos \theta \tan \alpha}, \quad (21)$$

where $\mathbf{u}$ completes a right-handed basis. Texture coordinates are

$$(u, v) = \left(\frac{1}{2} + \frac{1}{2} f u', \frac{1}{2} + \frac{1}{2} f v'\right), \quad (22)$$

with framing factor $f = 1$ for `ProjectedTextureMode=fill` (square sides on the cone; corners clipped) and $f = \sqrt{2}$ for `fit` (square inscribed in the cone). Samples with $(u, v) \notin [0, 1]^2$ contribute zero (hard mask). Cubemap and spherical modes ignore the cone and evaluate over 4π .

10.4 Cubemap direction mapping

Entity angles define a light-local frame $(\mathbf{r}, \mathbf{u}, \mathbf{b})$ (right, up, beam). A world direction ω maps to Source cube axes (Z-up) by

$$\mathbf{d} = (\omega \cdot \mathbf{r}, \omega \cdot \mathbf{b}, \omega \cdot \mathbf{u}), \quad (23)$$

i.e. $(d_x, d_y, d_z) = (\text{right}, \text{forward}, \text{up})$. Face selection and face coordinates follow the Direct3D cubemap table (identical to hardware `texCUBE` / Source VTF layout):

$$\begin{aligned} +X : sc &= -d_z, tc = -d_y, & -X : sc &= +d_z, tc = -d_y, \\ +Y : sc &= +d_x, tc = +d_z, & -Y : sc &= +d_x, tc = -d_z, \\ +Z : sc &= +d_x, tc = -d_y, & -Z : sc &= -d_x, tc = -d_y, \end{aligned} \quad (24)$$

with major-axis magnitude m and

$$u = \frac{1}{2}(sc/m + 1), \quad v = \frac{1}{2}(tc/m + 1). \quad (25)$$

Faces clamp at edges. CPU and GPU bakers share this convention.

10.5 Composite intensity

Let L_0 be the attenuated spot intensity (photometric or cone), $\mathbf{P} \in [0, 1]^3$ the projected RGB sample, and w_{IES} the optional IES weight. The bake contribution uses

$$L = L_0 w_{\text{IES}} \mathbf{P} \quad (26)$$

(componentwise; missing IES or projection $\Rightarrow$ factor 1). When both IES and `ProjectedTexture` are set, IES acts solely as an angular mask on the projection. Evaluation is wired through classic gather, CPU `PathLi`, GPU `PathLi`, and static-prop lighting.

11 Denoising

Opt-in via `-pt_denoise`; select backend with `-pt_denoiser`.

11.1 OIDN (default)

Intel Open Image Denoise `RTLightmap` model on CPU. One thread-local filter per bake worker (pinned to avoid oversubscription). Charts smaller than 16^2 are edge-clamp padded.

11.2 OptiX

NVIDIA CUDA HDR denoiser (no albedo/normal guides). Shared serialised GPU resources; same 16^2 padding.

11.3 Sakai (statistical)

Sakai et al. (2024)–inspired Welch-gated repeated 5×5 filter using variance of the mean where available:

$$t = \frac{|L_i - L_j|}{\sqrt{\text{Var}(\bar{L}_i) + \text{Var}(\bar{L}_j) + (0.04L_{\text{ref}})^2 + \varepsilon}}. \quad (27)$$

If $t > 1.35$, neighbour weight is attenuated by $\exp[-1.5((t - 1.35)/1.35)^2]$. Runs $1 + \text{radius}$ passes, then two low-gradient 3×3 evening passes. `-pt_denoise_radius` / `-pt_denoise_strength` apply here (OIDN/OptiX largely ignore radius).

12 Static prop lighting

12.1 Classic speedups

Four-wide SSE direct gather; with `-gpu`, bounce rays that miss / hit sky skip the BSP lightmap walk.

12.2 Pathtrace props

Under `-pathtrace` + `-StaticPropLighting`:

- **Prop lightmap texels** — same spp / bounces / NEE as world.
- **Vertex lighting** — virtual per-triangle lightmap with edge subdivision `-pt_prop_vertgrid` G (default 4):

$$\text{\#samples/triangle} = \frac{(G + 1)(G + 2)}{2}. \quad (28)$$

Lattice samples scatter to the three vertices by barycentric weights; each vertex normalises by accumulated weight. Unlit verts inherit from nearest lit. $G = 0$ = old one-sample-per-vertex.

Override quality with `-pt_prop_samples` / `-pt_prop_bounces` (defaults: $\max(8, S/4)$ spp; bounces = world). Multi-LOD models share lit colours across LODs.

13 Lightmap sample quality

13.1 Edge pull

Default on (inset 0.5 luxels; range 0–2 via `-edgepull`). For chart width W , sample coordinate s is clamped to $[i, W - 1 - i]$ (similarly t). Moves edge samples off adjacent solids — thin walls, jambs, floor/wall scallops. `-noedgepull` restores stock positions.

13.2 Seam stitching

On by default after `FinalLightFace`. Coplanar faces that share an edge (VBSP splits) blend near-edge luxels toward the neighbour value at the same world position so both sides converge to a common average. Per lightstyle and bump layer; displacements skipped. Disable with `-nostitch`.

14 Reference integrator (pseudocode)

Algorithm 1 PathLi — one spp at a surface sample

Require: position $\mathbf{x}$, normal $\mathbf{n}$, bounce limit N , seed

```

1:  $\beta \leftarrow \mathbf{1}$ ;  $L \leftarrow \mathbf{0}$ ; depth  $\leftarrow N + 1$ 
2: for  $k \leftarrow 0$  to depth  $- 1$  do
3:    $L \leftarrow L + \beta \cdot \text{NEE}(\mathbf{x}, \mathbf{n}, k)$  ▷ soft only if  $k = 0$  or softmode=all
4:    $\omega \leftarrow \text{CosineHemisphere}(\mathbf{n})$ 
5:   trace closest hit along  $\omega$ , applying filter panes to  $\beta$ 
6:   if miss or sky then
7:      $L \leftarrow L + \beta \cdot L_{\text{sky}}(\mathbf{x})$ ; break
8:   end if
9:    $\beta \leftarrow \beta \cdot \rho_{\text{hit}}$  ▷ or spectral Reflect( $\lambda$ )
10:  if  $k \geq 1$  then
11:    Russian roulette on  $\beta$ 
12:  end if
13:   $\mathbf{x}, \mathbf{n} \leftarrow$  hit point / oriented normal
14: end for
15: return firefly-clamp( $L$ )

```

15 Parameter cheat sheet

Flag	Default	Role
-pathtrace	off	Enable DXR baker
-pt_gpu/-pt_cpu	GPU	Integrator backend
-pt_samples S	quality	spp per luxel
-pt_bounces N	3 (1 fast)	Indirect hops; 0=direct+sky
-pt_aa A	3	Luxel footprint grid
-pt_spectral	on	Hero- λ GPU transport
-pt_lights K	0=all	Local NEE samples (bounce > 0)
-pt_emit_samples	64	Area-emit NEE draws (0=all)
-pt_lightradius	0	Soft disk for point/spot
-pt_lightpenumbra	1	CHSS scale
-pt_softsamples	16	Soft ray cap
-pt_softmode	direct	Soft at luxel / all vertices
-pt_denoise	off	Enable denoise
-pt_denoiser	oidn	oidn optix sakai
-pt_prop_vertgrid	4	Prop vertex lattice
-bounce N	100	Classic radiosity rounds (0=direct)
-texbounce	off	Textured radiosity albedo
-energy/-cavity	off / 0.70	Cavity damp
-bounce_boost	1	Artistic bounce scale
-bounce_chroma	0	Early-bounce Oklab sat
-bounce_weld	off	Coplanar edge GI weld
-gpu	off	OpenCL classic gather
-ao	off	Hemisphere AO pass
-edgepull	0.5	Luxel edge inset
-nostitch	—	Disable seam stitch

Config presets

Hammer-friendly flag sheets live under `configs/` and load via `-config full` (searches `.cfg/.txt` next to the binary). Do not put `-game /` map path inside the config file.

16 Implementation map

Module	Responsibility
<code>pathtrace_dxr.cpp</code>	CPU PathLi, bake orchestration, AA, cleanup
<code>pathtrace_dxr_gpu_bake.inl</code>	HLSL PathLi (spectral, NEE, filters)
<code>pathtrace_dxr_device.*</code>	D3D12 / DXR device, AS, dispatches
<code>pt_spectral.h</code>	Smits bases, CIE CMFs, RGB $\leftrightarrow\lambda$
<code>oklab.h</code>	Perceptual colour ops
<code>vrاد_emit*.cpp</code>	Textured emission + area mesh
<code>vrاد_filter.cpp</code>	Colored glass transmittance
<code>vrاد.cpp / vismat.cpp</code>	Classic radiosity loop
<code>radial.cpp</code>	Bounce splat / weld
<code>vrاد_gpu.*</code>	OpenCL BVH + gather
<code>ao.cpp</code>	Multi-scale AO
<code>absorb.cpp / bounce_vol.cpp</code>	Volume effects
<code>ies_profile.*</code>	IESNA LM-63 load + angular eval
<code>light_projection.*</code>	Projected VTF load, CPU sample, GPU arrays
<code>pathtrace_*denoise*</code>	OIDN / OptiX / Sakai
<code>vrادstaticprops.cpp</code>	Prop SSE + pathtrace vertgrid
<code>lightmap.cpp</code>	Edge pull, final face lighting, spot parse

References & provenance

- Kajiya, J. T. — The rendering equation (SIGGRAPH 1986).
- Veach, E. — Robust Monte Carlo methods for light transport (PhD, 1997): power-heuristic MIS.
- Smits, B. — An RGB to spectrum conversion for reflectances (JGTools 1999); PBRT-v3 tabulated bases.
- Wyman, C. et al. — Analytic CIE 1931 CMF fits.
- Ottosson, B. — Oklab (2020).
- Pharr, Jakob, Humphreys — *Physically Based Rendering* (area lights, spectra).
- Sakai et al. — Statistical lightmap filtering (2024 inspiration for the Sakai denoiser).
- Intel Open Image Denoise — RTLightmap model.
- NVIDIA OptiX — HDR AI denoiser.
- IESNA LM-63 — standard file format for luminous intensity distributions.
- Microsoft Direct3D — cubemap face selection and (s, t) mapping (`texCUBE` / environment maps).
- Valve Source SDK 2013 — stock VRAD radiosity / lightmap pipeline; VTF cubemap face order (`CUBEMAP_FACE_*`).

Algorithms are described as implemented in the CustomVRAD source tree. CLI defaults and clamps may change between revisions; `vrad.exe help` output and `configs/full.cfg` define shipping presets.